THE JS EVENT LOOP

Guess the Output — 10 Challenges

Synchronous code • Microtasks (Promises) • Macrotasks (setTimeout & Events)

The Rule Behind Every Question

CALL STACK

Runs your synchronous code top to bottom, line by line — first.

→

MICROTASK QUEUE

Promise .then() / async-await continuations. DRAINS COMPLETELY — even new ones added mid-drain.

→

MACROTASK QUEUE

setTimeout, setInterval, click / event listeners. Only ONE runs, then we check microtasks again.

↻ *after ONE macrotask runs, the engine checks the microtask queue again before taking the next macrotask*

THE GOLDEN RULE

1. Run ALL synchronous code first (top to bottom).
2. When the call stack is empty, drain the ENTIRE microtask queue — Promises, async/await — even ones created along the way.
3. Take exactly ONE task from the macrotask queue (setTimeout, a click, etc.) and run it.
4. Go back to step 2. Repeat forever.

How to Play

1

Read the code carefully

Every question mixes sync code, Promises, setTimeout, and sometimes event listeners.

2

Write down your guessed output — in order

Don't just say what prints. Say in WHAT ORDER it prints.

3

Flip to the answer slide

Check the color-coded trace: grey = synchronous, purple = microtask, orange = macrotask.

4

Read the reasoning

Each answer explains WHY — tying back to the Golden Rule on the previous slide.

Q1 Warm-up: setTimeout(0)

```
console.log("A");

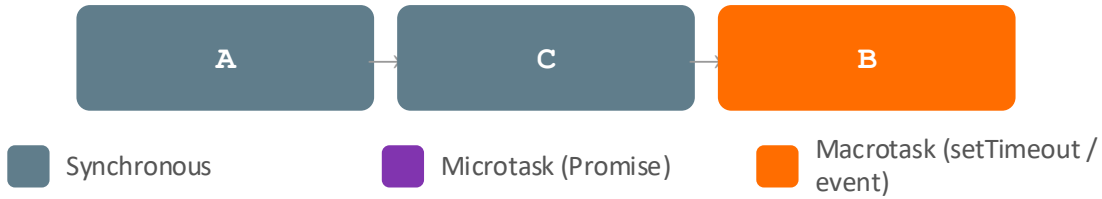
setTimeout(() => console.log("B"), 0);

console.log("C");
```

❓ ***What gets printed to the console, and in what order?***

Q1 Answer — Warm-up: setTimeout(0)

Output order:



- The call stack always finishes ALL synchronous code first — that's A, then C.
- setTimeout(fn, 0) does NOT mean "run immediately" — it means "queue this in the macrotask queue, to run once the stack is empty."
- So B is delayed to the very end, even with a 0ms delay.

Q2 Promise vs setTimeout

```
console.log("1");

setTimeout(() => console.log("2"), 0);

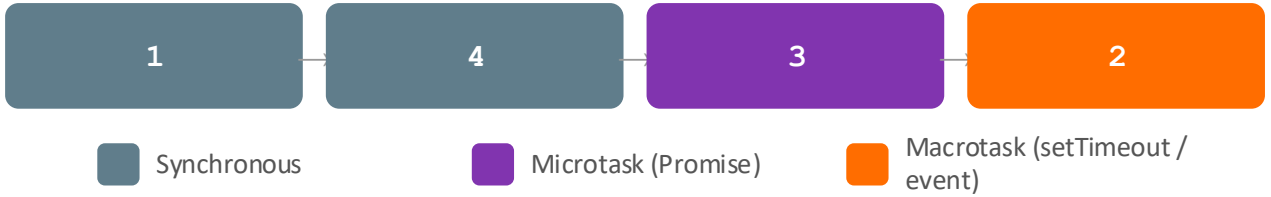
Promise.resolve().then(() => console.log("3"));

console.log("4");
```

❓ *What gets printed to the console, and in what order?*

Q2 Answer — Promise vs setTimeout

Output order:



- Sync code always wins first: 1, then 4.
- Once the stack is empty, the MICROTASK queue is drained completely before anything else — so the Promise's .then() ("3") runs next.
- Only after every microtask is done does the engine take ONE task from the macrotask queue — the setTimeout ("2") runs last.

Q3 Chained .then() — do BOTH beat setTimeout?

```
console.log("start");

setTimeout(() => console.log("timeout"), 0);

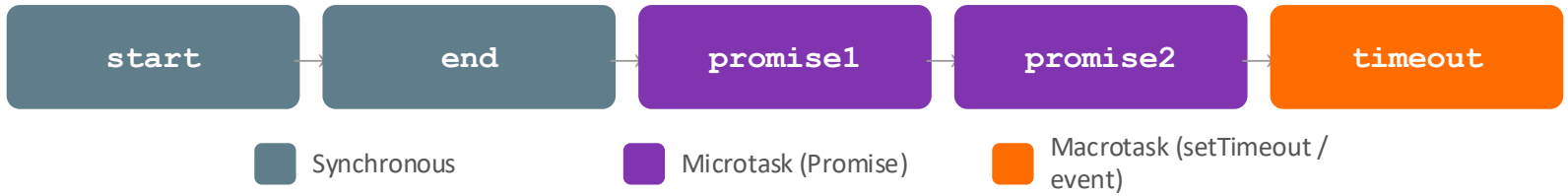
Promise.resolve()
  .then(() => console.log("promise1"))
  .then(() => console.log("promise2"));

console.log("end");
```

❓ *What gets printed to the console, and in what order?*

Q3 Answer — Chained .then() — do BOTH beat setTimeout?

Output order:



- Sync: start, end.
- "promise1" runs first from the microtask queue — and this itself queues "promise2" as a NEW microtask.
- Key rule: the engine keeps draining the microtask queue until it's truly empty — even microtasks added DURING draining. So promise2 also runs before any macrotask.
- Only after both promises are done does "timeout" (macrotask) get its turn.

Q4 Two setTimeouts + one Promise

```
setTimeout(() => console.log("timeout1"), 0);
setTimeout(() => console.log("timeout2"), 0);

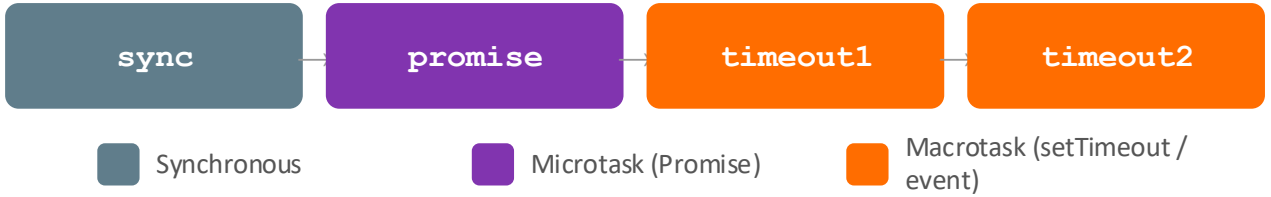
Promise.resolve().then(() => console.log("promise"));

console.log("sync");
```

❓ *What gets printed to the console, and in what order?*

Q4 Answer — Two setTimeouts + one Promise

Output order:



- "sync" logs immediately — it's the only synchronous console.log here.
- The microtask queue (the Promise) always drains before ANY macrotask, so "promise" comes next.
- The macrotask queue is First-In-First-Out — timeout1 was registered before timeout2, so it fires first.

Q5 async / await — the hidden .then()

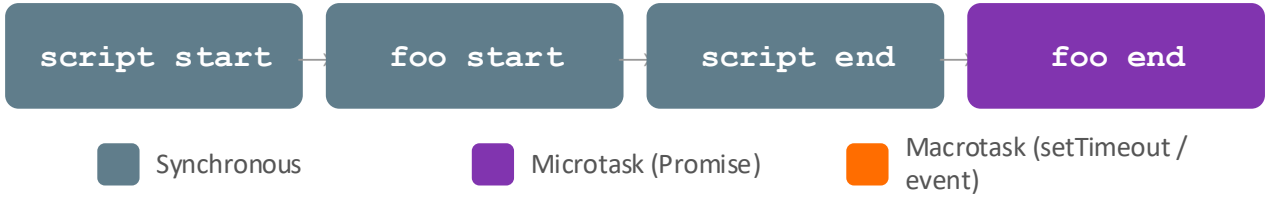
```
async function foo() {
  console.log("foo start");
  await null;
  console.log("foo end");
}

console.log("script start");
foo();
console.log("script end");
```

❓ *What gets printed to the console, and in what order?*

Q5 Answer — async / await — the hidden .then()

Output order:



- Calling foo() runs it immediately (synchronously) up until the first 'await' — so "foo start" happens right in the middle of the sync code.
- 'await' pauses the function and schedules everything AFTER it as a microtask — control returns to whoever called foo().
- So "script end" (sync) finishes before "foo end" (microtask) gets its turn.
- Mental model: 'await x' behaves almost exactly like 'x.then(restOfFunction)'.

Q6 Click handler vs Promise vs setTimeout

```
btn.addEventListener("click", () => console.log("clicked"));

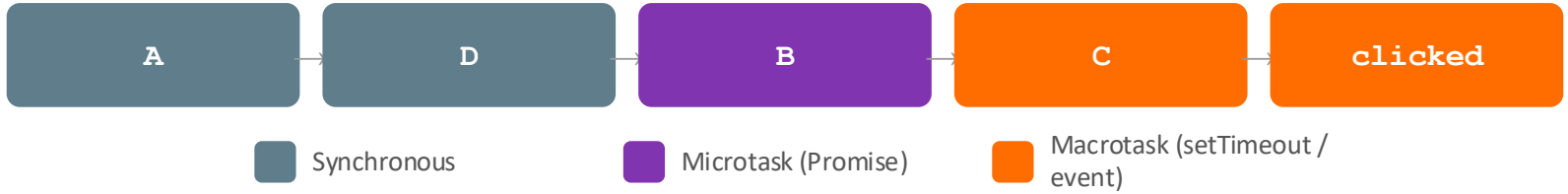
console.log("A");
Promise.resolve().then(() => console.log("B"));
setTimeout(() => console.log("C"), 0);
console.log("D");

// Assume the button is clicked several seconds later —
// well after the timeout above has already fired.
```

❓ *What gets printed to the console, and in what order?*

Q6 Answer — Click handler vs Promise vs setTimeout

Output order:



- A real user click is NOT instant to the engine — it's queued as a macrotask, exactly like setTimeout.
- Sync first: A, D. Then the microtask (B) drains completely.
- C fires before "clicked" only because we assumed the click happens seconds later — both are macrotasks, and macrotasks run strictly in the order they actually arrive.
- This is the direct answer to 'is addEventListener async?' — yes, and it queues in the SAME macrotask queue as setTimeout.

Q7 A setTimeout created INSIDE a .then()

```
console.log("1");

Promise.resolve().then(() => {
  console.log("2");
  setTimeout(() => console.log("3"), 0);
});

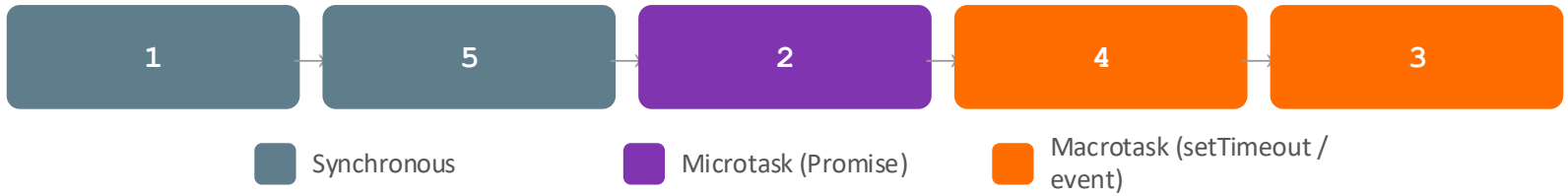
setTimeout(() => console.log("4"), 0);

console.log("5");
```

❓ *What gets printed to the console, and in what order?*

Q7 Answer — A setTimeout created INSIDE a .then()

Output order:



- Sync: 1, 5.
- Microtask queue drains: "2" runs. While it runs, a NEW setTimeout ("3") gets added to the macrotask queue — but only NOW, during the microtask phase.
- Macrotask queue is FIFO by the time each timer was registered: timeout "4" was registered during the original sync code — BEFORE timeout "3" was registered (which only happened during the microtask phase). So 4 goes first.
- This proves microtasks can create new macrotasks, but they still have to wait their turn at the back of the macrotask line.

Q8 Microtasks spawning MORE microtasks

```
console.log("A");

setTimeout(() => console.log("B"), 0);

Promise.resolve().then(() => {
  console.log("C");
  Promise.resolve().then(() => console.log("D"));
});

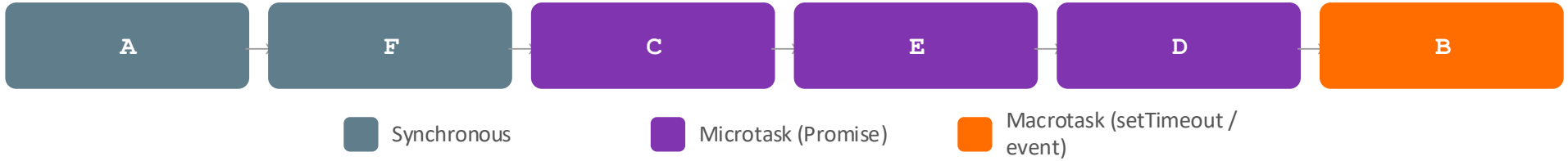
Promise.resolve().then(() => console.log("E"));

console.log("F");
```

❓ ***What gets printed to the console, and in what order?***

Q8 Answer — Microtasks spawning MORE microtasks

Output order:



- Sync: A, F.
- Microtask queue starts as [→C-block, →E]. First C-block runs, logging "C" — and it queues a brand-new microtask ("D") onto the END of the SAME queue.
- The queue isn't empty yet (E is still waiting), so the engine keeps draining: "E" runs next, THEN "D" (since D was queued after E).
- Only once the microtask queue is truly, completely empty does the engine move to the macrotask queue: "B" runs last.
- This is the clearest proof that ALL microtasks — even ones created while draining — finish before the next macrotask.

Q9 Gotcha: btn.click() called by YOUR code

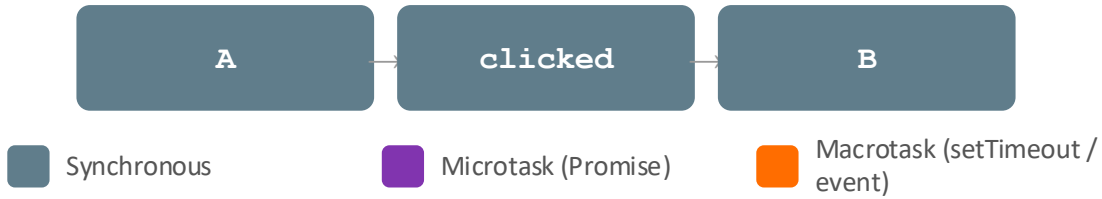
```
btn.addEventListener("click", () => console.log("clicked"));

console.log("A");
btn.click();   // triggered programmatically, not by the user
console.log("B");
```

❓ *What gets printed to the console, and in what order?*

Q9 Answer — Gotcha: btn.click() called by YOUR code

Output order:



- This is the exception every student should know: calling element.click() in YOUR code fires the listener SYNCHRONOUSLY, in the same call stack — it does NOT get queued.
- That's different from a real user physically clicking the button, which the browser always queues as a macrotask (see Q6).
- Rule of thumb: it's not 'addEventListener is async' — it's 'the EVENT DISPATCH decides sync vs async'. A genuine user/browser-triggered event is async; a script calling .click()/.dispatchEvent() runs it inline.

Q10 Final : everything together

```
console.log("1");

btn.addEventListener("click", () => console.log("2 (click)"));

async function run() {
  console.log("3");
  await Promise.resolve();
  console.log("4");
}

setTimeout(() => console.log("5"), 0);

run();

Promise.resolve().then(() => console.log("6"));

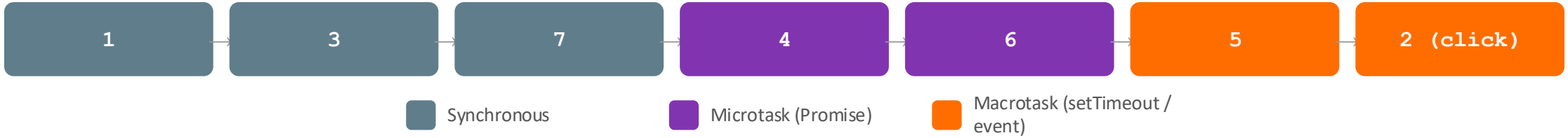
console.log("7");

// Assume the button is clicked several seconds later,
// well after everything above has finished.
```

❓ ***What gets printed to the console, and in what order?***

Q10 Answer — Final Boss: everything together

Output order:



- Sync run: "1" logs. run() is CALLED, which synchronously logs "3" before hitting 'await' — then pauses. Back in the main script: "7" logs last.
- Microtask queue order matters: run()'s 'await' queued its continuation ("4") FIRST, before the explicit Promise.resolve().then() ("6") was even reached — so "4" runs before "6".
- Once microtasks are empty, the macrotask queue runs: the setTimeout ("5") fires — it was scheduled minutes before the click.
- The click, happening seconds later in real time, is simply the last macrotask to arrive — "2 (click)" is last.

You've Got the Event Loop ✅

Sync first. Microtasks drain completely. One macrotask at a time. Repeat.

addEventListener callbacks ARE asynchronous — they queue in the macrotask queue, just like setTimeout. That's why we still call them a different flavor of async than Promises, which get microtask priority.